

Open Atomic Ethernet

Making A New Ethernet

David Johnston (ex Intel)

OCP-OAE Plenary, #26
02025-JUN-25

[https://opencompute-org.zoom.us/j/89756960348?
pwd=vKwSvuFHZSCQsVEpHI36fAJQCeXez8.1](https://opencompute-org.zoom.us/j/89756960348?pwd=vKwSvuFHZSCQsVEpHI36fAJQCeXez8.1)

<https://www.opencompute.org/projects/open-atomic-ethernet>

https://www.opencompute.org/w/index.php?title=Open_Atomic_Ethernet

<https://github.com/opencomputeproject/ocp-odei>

Open Atomic Ethernet

Making A New Ethernet

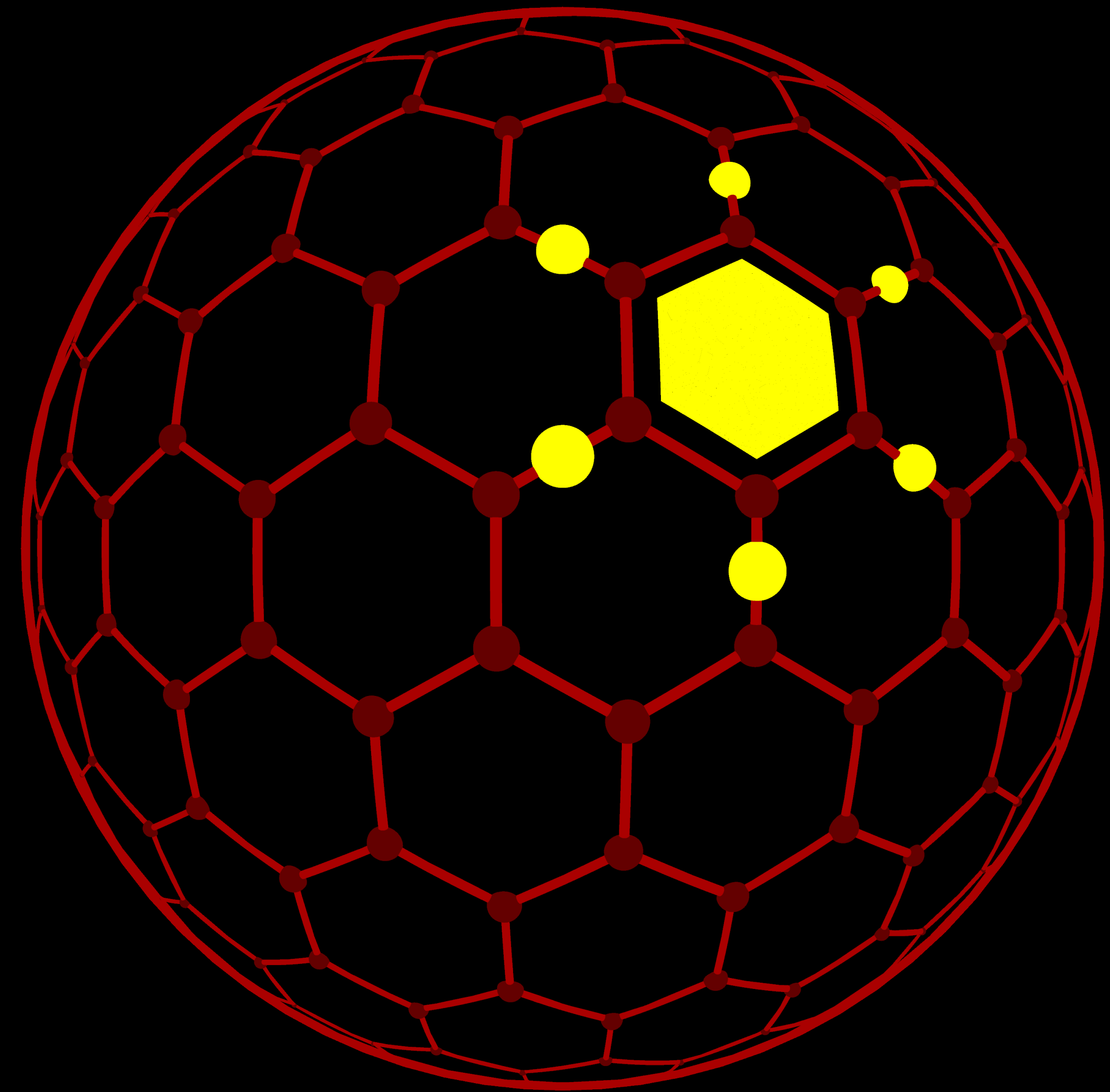
David Johnston (ex Intel)

OCP-OAE Plenary, #26

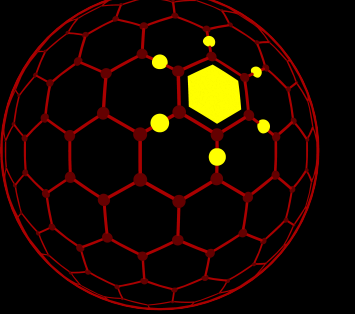
02025-JUN-25

[Zoom Link](#)*

*[https://opencompute-org.zoom.us/j/89756960348?
pwd=vKwSvuFHZSCQsVEpHI36fAJQCeXez8.1](https://opencompute-org.zoom.us/j/89756960348?pwd=vKwSvuFHZSCQsVEpHI36fAJQCeXez8.1)



David Johnston



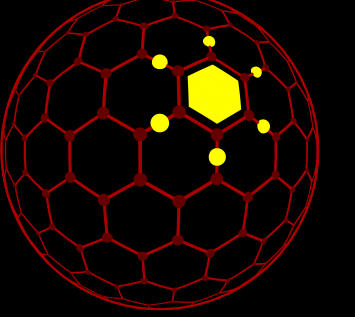
David brings a unique blend of deep technical rigor, industry experience, and standards leadership to our discussion today. His talk will walk us through the real-world process of developing a new Ethernet standard—from forming an IEEE 802 working group, to writing and balloting specifications, amendments, and navigating the intricacies of the 4-year maintenance cycle.

But this won't just be a policy walkthrough. David will take us all the way down to the physics of individual Ethernet links—link security, electrical signaling, energy efficiency at the level of picojoules per bit, and even the fundamental constraints of photon detection and energy use in optical media. Expect a lively comparison of electrical vs. optical Ethernet from someone who's worked at every level of the stack—from quantum randomness to protocol design.

BIO: David spent over two decades at Intel, where he served as a Principal Engineer and became best known for his groundbreaking work on Random Number Generators, including the Intel RdRand and RdSeed instructions. He literally wrote the book on this: *Random Number Generators: Principles and Practices*. He also co-authored *Designing to FIPS 140* and was a primary contributor to the CMUF Entropy Working Group, helping guide RNG standards at NIST. He is currently the founder of Primitive Security, a consulting firm specializing in secure hardware design, entropy validation, and FIPS certification. Ask him about the background image on his slides—it's not just decorative; it's a visual representation of online entropy testing, which David recently presented at SPACE 2024 in India.

David's contributions to IEEE 802 span 802.11, 802.16, 802.1, and 802.3—especially around link-layer security and randomness requirements. His fingerprints are on standards from FIDO to USB, and even proprietary Intel protocols. Earlier in his career, David built test equipment, tracking devices, and wireless chipsets with companies like Cadence/Tality and Mobilian, and worked on electronics for Formula 1 cars at Pi Research. He started his journey at Inmos in the UK, working on supercomputer modules fresh out of college.

Dean Gladish



Dean Gladish is a software engineer and a Master's student at Georgia Tech in Machine Learning. His journey in tech started at Carleton College with a degree in Mathematics, which gave him a solid foundation in the abstract thinking for over three years, immersed in the world of software development as the Lead Web App Developer at Qooley(.Vercel.App) and at AlphaScenario.com, a corporate events marketplace. There, I was responsible for about 90% of the development and maintenance of our React-based website. It was a role that demanded quick learning and adaptability, as I handled everything from the front-end to the back-end and eventually built out features like artist cards, price filtering, and photo galleries that give me a unique perspective, on problem-solving. It's not just about the code; it's about understanding the “why” behind it and how it impacts people.

Dean is fascinated by the fundamental principles of computation and computational complexity and how they can be applied to knowledge graphs like the NSF-funded [CollabNext.io](#) or the Wolfram Institute's research on multi-way tag systems; now, he is excited to bring his experience and enthusiasm to the Open Atomic Ethernet project, contributing to the development of a more reliable and efficient networking standard.

Take Aways

Ethernet Was Designed for “Best Effort”

- Atomicity cannot be guaranteed without the Alternating Bit Protocol (ABP)
- ABP requires a round trip (RT) (within each link individually)
- The RT *does not* need to be in series: Alice —link—> Bob —link—> Alice
- Acknowledges are flits, flits can be 8-byte slices in the SerDes
- There is NO bandwidth degradation if the occupation time on the wire of the “transfer unit” is *longer* than the cable.
- PROPOSAL (Bill Lynch): An “alternation” field of $\log_2(k)$ bits.
- Existing Ethernet assumes multiple packets in flight before needing Ack
- Atomic Ethernet “requires” each packet to be acknowledged (signal)

Is Ethernet Atomic?

Robert Garner:

- In my work on the history of the emergence of the Ethernet: “I've learned that the 1968 ALOHA guys implemented an acknowledgement protocol based an alternating 0/1 bit. It was based on a paper by Bill Lynch at Case Institute (who later joined Xerox for the STAR OS called Pilot with XNS). Bill's (and ALOHA's) objective was to detect dropped packets over noisy telephone/radio connections, paper attached.
- However, such an acknowledgement protocol throttles performance. ALOHA only supported one outstanding packet at a time on the (9.6-kbps) channel (an 80-character terminal line being sent to the central IBM 360, or a return line from the 360 to a terminal.)
- In his nascent April 1973 "ALTO ALOHA" memo, Bob assumed that every packet would be ACK'd. However, his rough calculations showed that the throughput, especially for small packets, sucked, down by 3/4 to 1/2 to 1/5. Thus ACKs didn't survive to his founding May 22, 1973 “Ether Acquisition” memo. For highest pipelined performance, you really need to support multiple outstanding packets in the channel. (Unless what you actually have is a point-to-point connection or short bus.) A higher-level protocol like PUP or TCP can then acknowledge the receipt of multiple packets all at once, or ask for retransmission when one or more are missing”

Story So Far

I heartily concur with all the comments made by Bartlett, Scantlebury, and Wilkinson concerning the transmission of information

- Bob Metcalfe concluded in his PhD Thesis that it was necessary to push more than one packet onto the wire otherwise bandwidth would be inefficiently used. He traded off reliability (best effort) in favor of bandwidth efficiency. This seemed to provide a one-way Shannon Channel without the need for acknowledgements.
- Bill Lynch* proved that 2 bits were necessary: a verify bit and an alternation bit, for reliable transmission over noisy channels, as each addresses distinct aspects of the problem: error detection and sequencing. The alternation bit inherently requires a round-trip acknowledgment (waiting for the alternating bit) because it depends on feedback from the receiver to toggle states safely and ensure proper sequencing of messages. **Omitting either mechanism compromises safety, making it impossible to achieve robust communication.**
- Bartlett, Scantlebury and Wilkinson in a 1968/69 note showed that Bill Lynch's proof was incorrect, and only 1 bit was needed -- the alternating bit -- **which requires a round trip to complete transmission.**
- Bill Lynch had no memory of the Bartlett paper (or his own reply) until it was brought to his attention. Bill adds:
 - I'd like to reinforce some of the Bartlett comments. Optimizing these algorithms certainly does depend on what error model is being used. The whole impetus for my investigation was the fact that the performance of the link was dominated by strong, sporadic, fractional second, switching noise coupled into a POTS that neither God nor man intended to carry data. (The usual assumption of Gaussian noise wasn't adequate.) This noise burst often lasted through a packet transmission and then the transmission of the ACK/NAK from the previous transmission and blew away any independence assumption. I don't expect to see this kind of noise in today's comm lines.
 - Secondly, I often thought of the alternation bit as a mod 2 counter. It represented the $\text{mod } 2$ function applied to ordinal number of the input packet. This point of view can be usefully extended to the mod k function applied to the ordinal of the input, giving an "alternation" field of $\log_2(k)$ bits or so. This would allow the received state of up to the last k packets to be known and provide a correction basis for a multi-hop pipeline of links up to K in length (think TCP here). Among other things it shows how to validate the Bartlett conjecture that a scheme could be constructed that would only fail with too many (more than

- Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines. Bill Lynch, 1968
- Defines Alternating Bit Protocol
- Defines Exactly Once Semantics
- Proves that the alternating bit is necessary
- Requires round trip



Computer Systems

R. L. ASHENHURST, Editor

Reliable Full-Duplex File Transmission over Half-Duplex Telephone Lines

W. C. LYNCH
Case Institute of Technology, Cleveland, Ohio

A field-proven scheme for achieving reliable duplex transmission over a half-duplex communication line is presented, and to demonstrate the difficulty of the problem, another similar scheme, which is only slightly unreliable, is also presented. A flowchart for the reliable scheme and some interesting examples are given.

KEY WORDS AND KEY PHRASES: telephone communication, half duplex, transmission, error correction, full duplex, telephone errors
CR CATEGORIES: 3.81, 4.41, 6.35

1. Introduction

Since data transmission over telephone lines has been an accomplished fact for many years, one would assume that practical and adequate techniques for the control of errors in this noisy medium would have been worked out and promulgated. Confidence in such an assumption appears to be ill-placed with respect to products offered by the major computer manufacturers. The standard hardware and software packages seem to adequately detect errors, but the automatic correction of such errors is quite another matter.

The purpose of this article is to describe an adequate scheme for both the detection and correction of transmission errors over a half-duplex telephone line. This is a difficult problem due to the necessity of sending control, error, and verification information over the same noisy line that first caused an error.

2. Half-Duplex Lines

The transmission scheme presented here is with reference to standard voice grade telephone lines. Such lines may be either a dial-up switched line transmitting at 2000 baud (bits/second) using a 201A modem, or a 2400 baud hard wire line using a 201B modem.

Such lines are called half-duplex because they may transmit in either direction, one direction at a time. By contrast, a simplex line may transmit only one way, while

a (full-) duplex line may transmit both ways at the same time. An unpleasant feature of half-duplex lines is the time required to switch directions. A reversal time on the order of 200msec is required to allow echoes to die out. This reversal time significantly complicates the process of maximizing the effective (error-free) data transmission rate.

To avoid confusion it should be observed that the terms simplex, half-duplex, and duplex apply independently at each level of organization. Thus we may have full-duplex transmission of files while the records of that file are interchanged on a half-duplex basis. Simplex file transmission may occur on a full-duplex line. Descriptions of transmission schemes are often inexplicit on this point (e.g. half duplex typically means half duplex by *file*, not record).

3. Reliable Simplex Transmission

The transmission scheme is first presented so as to form a simplex channel, i.e. it will pass data in only one direction. Then we extend the algorithm to cover duplex file transmission (both ways at once).

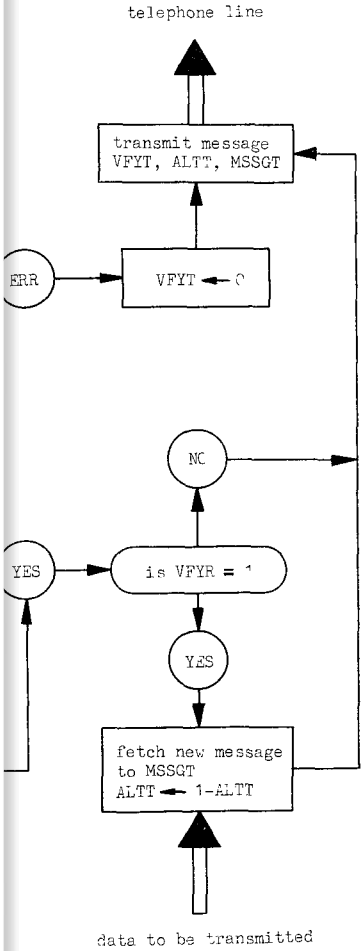
It is assumed throughout that the system has perfect error detection. To be more precise, if a message is sent from A to B, B will either miss the message completely (*drop* the message), or if it does not drop the message, will be able to determine whether or not the message is in error. For messages on the order of dozens of characters in length, a horizontal and vertical parity scheme similar to that employed on magnetic tape seems to be more than adequate. Odd parity should always be used in order to allow the modems to resynchronize themselves.

It is further assumed that there are no message drops. Later in the paper the algorithm is extended to cover the message drop case, and thus lift this restriction.

To each message sent from A to B we attach an extra bit called the alternation bit. This bit is of course subject to the error checking. After B receives the message it decides if the message had no errors (is error-free). It sends back to A a verification message (*verify bit*) indicating to A whether or not the immediately preceding A → B message was error-free. After A receives this verification one of three possibilities holds: (1) the A → B was good, (2) the A → B message was bad, (3) the verification was in error so that A does not know whether the A → B message was good or bad. In cases (2) and (3) A simply resends the same A → B message as before. In case (1),

it is possible for a terminal to, at any time, treat an error-free message as in error. This tactic may be useful, e.g., the receiving terminal is in error and has no place to file the message.

One of the problems in the system are message synchronization. A terminal does not have to respond with the verification message until any amount of time before the next response could allow for the danger of another message. The problems in the system are not dropped (a condition which we call a hang up).



can be accommodated in a file. The print speeds of 150-180 lines per minute make the transmission consists of a single message.

One problem in constructing adequate verification schemes, I will present and discuss, is the one proposed by computer manufacturers in a half-duplex alternating message scheme, as is the one proposed by each message containing only one bit, as in Section 4, each

terminal B				
msg	err	verfy	alt	accept
AB1	OK	1	1	YES
AB2	OK	1	0	YES
(AB2)	ERR	(0)	(0)	--
(AB2)	ERR	(1)	(0)	--
AB2	OK	1	0	NO
(AB3)	ERR	(1)	(1)	--
(AB3)	ERR	(0)	(1)	--
AB3	OK	1	1	YES
(AB3)	ERR	(0)	(1)	--
AB3	OK	1	1	NO
AB4	OK	1	0	YES

exchange on a noisy line

terminal B		
msg	err	ack
AB1	OK	1
AB2	OK	1
AB2	OK	0
AB3	OK	1

exchange in the second scheme

the task, we assumed we had several available commercial modems with the analog equipment manufacturer; we found several modems with good accuracy, resolution, control part of the display, and, completely happy with the results of the commercial systems we now to couple the display to our system. We had serious doubts about the reliability of the parent computer system.

the Advanced Research Projects Agency (DARPA) contract 627, Amendment No. 2, and contract 9(628)-5965, Air Force Cambridge Research Laboratory, United States of Massachusetts 01730. The research was conducted at the Massachusetts Institute of Technology, Cambridge, Mass.

Volume 11 / Number 6 / June, 1968

Communications of the ACM 407

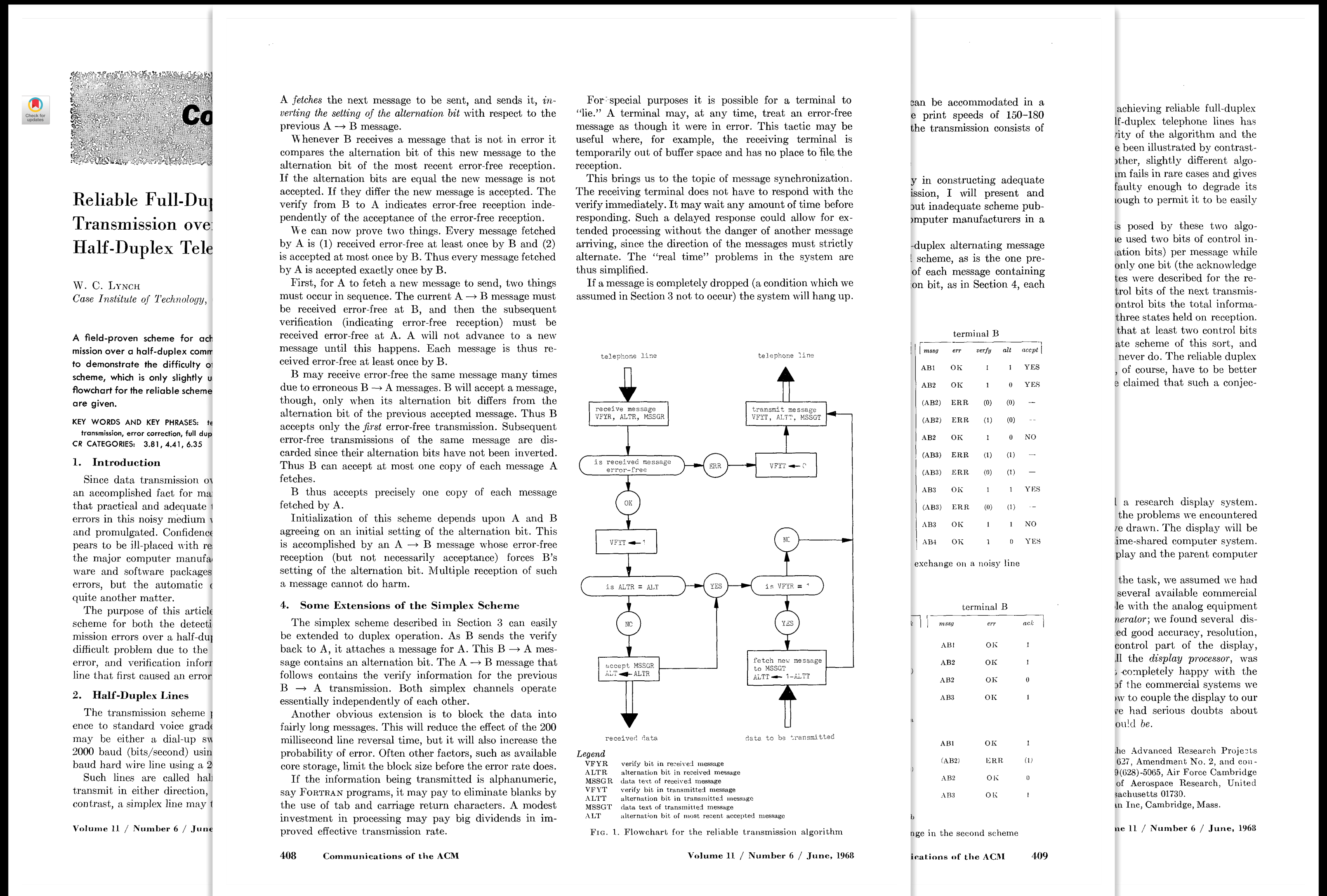
Volume 11 / Number 6 / June, 1968

Communications of the ACM 409

*ACM: [Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines](#).

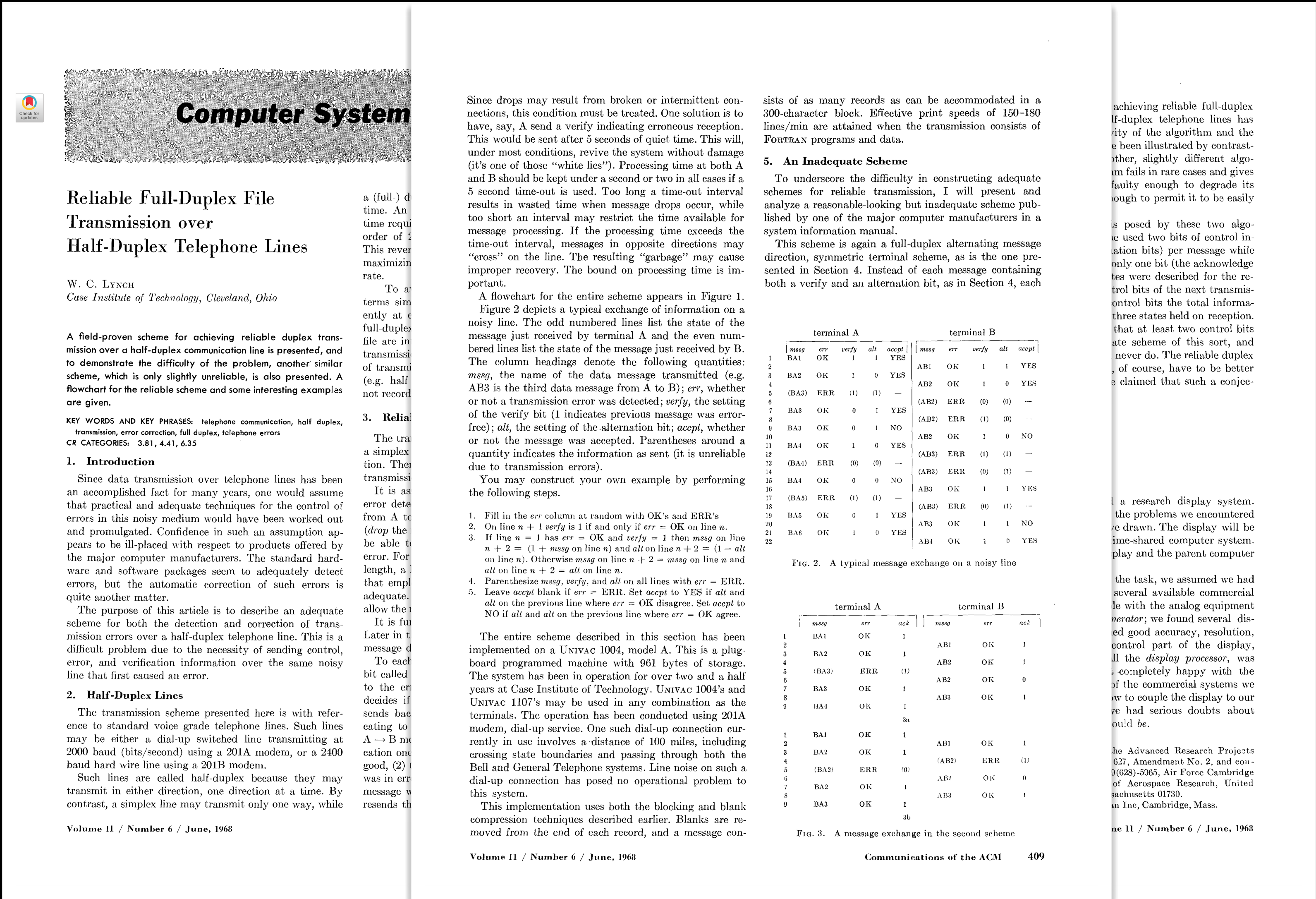
Is Ethernet Atomic?

- Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines. Bill Lynch, 1968
- Defines Alternating Bit Protocol
- Defines Exactly Once Semantics
- *Proves that the alternating bit is necessary*
- *Requires round trip*



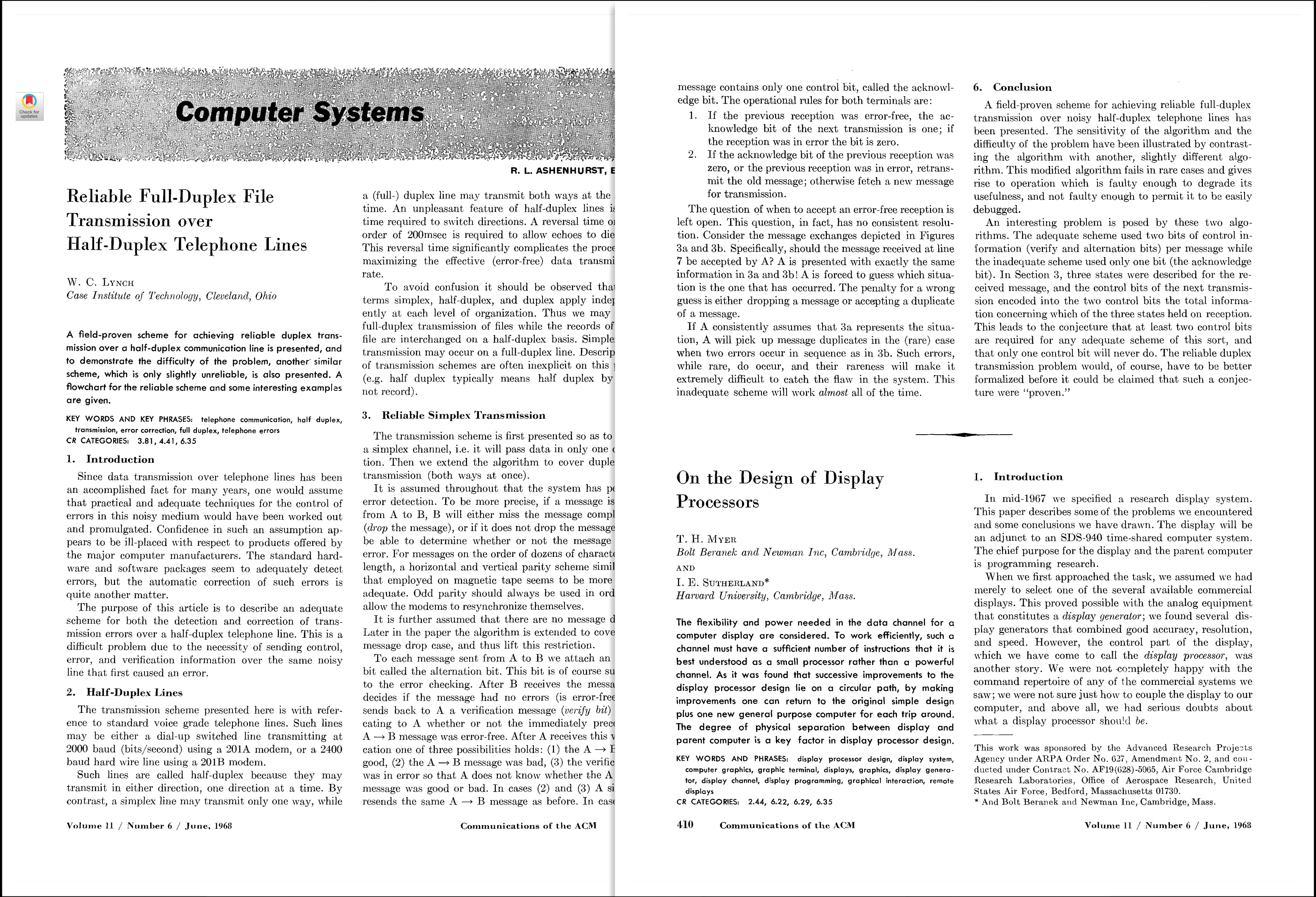
Is Ethernet Atomic?

- Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines. Bill Lynch, 1968
- Defines Alternating Bit Protocol
- Defines Exactly Once Semantics
- Proves that the alternating bit is necessary
- Requires round trip



Is Ethernet Atomic?

- Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines. Bill Lynch, 1968
- Defines Alternating Bit Protocol
- Defines Exactly Once Semantics
- Proves that the alternating bit is necessary
- Requires round trip



- Proves that only one bit is necessary — the alternating bit, which requires a round trip on every packet ...
- This is a Forward-in Time only (FITO) way to look at protocols, typical of engineers at the time (and now)
- Reversible protocols need 2+ bits, to indicate the direction of evolution of the state machine



Is Ethernet Atomic?

- Bill Lynch: Both the verify bit and alternation bit are necessary for reliable transmission over noisy channels, as each addresses distinct aspects of the problem: error detection and sequencing. The alternation bit inherently requires a round-trip acknowledgment (waiting for the verify bit) because it depends on feedback from the receiver to toggle states safely and ensure proper sequencing of messages. Omitting either mechanism compromises reliability, making it impossible to achieve robust communication.
- Bartlett et al: Proves Only the alternating bit is necessary:
 - The Alternating Bit.
- Bill Lynch Agrees in response to Bartlett, and continues (in Email Dialog)

Potential degradation/throttling of throughput do to acting every packet

Potential degradation/throttling of throughput due to ACKing every packet (with only one packet outstanding), even with 400 Gb/s Ethernet. However, such an acknowledgement protocol throttles performance. ALOHA only supported one outstanding packet at a time on the (9.6-kbps) channel (an 80-character terminal line being sent to the central IBM 360, or a return line from the 360 to a terminal.)

It only throttles performance if the round trip delay is longer than the length of the packet or one packet on the wire. At 25Gb or 100Gb (per lane) the in-flight packet length is still longer than a short inter-rack link.

You may actually be in that throttling realm (with round trip delay + the ACK duration). Assuming 400 Gb/s, 1518-byte standard-max Ethernet data packets and 64-byte standard Ethernet ACKs, and a 15-nanosecond propagation delay (~ 3-meter electrical cable), the minimum transmission time (round-trip delay + the ACK duration) is roughly 31.2 nanoseconds. Compare that to the data packet duration of 120 nanoseconds. As a result, you're losing half the throughput to the ACKs, or 49% efficiency. (see below for calculations)

Using 9216-byte jumbo frames, with a 15-nanosecond propagation delay, the efficiency is better at 85%. (But jumbo frames may invite a latency issue?)

One way around this is to support multiple outstanding packets with corresponding ACKs and queues. But what of KISS (“Keep It Simple Stupid”)?

(Wherein lies the boundary between sky-high idealism and down-to-earth practicality?)

Any protocol which is not intimately paired with it's acknowledge “works *almost* all the time”. (From Bill Lynch)

See Paper to be published on the
OAE Wiki

Conclusions in Bill Lynch paper*

- *“A field-proven scheme for achieving reliable full-duplex transmission over noisy half-duplex telephone lines has been presented. The sensitivity of the algorithm and the difficulty of the problem have been illustrated by contrasting the algorithm with another, slightly different algorithm. This modified algorithm fails in rare cases and gives rise to operation which is faulty enough to degrade its usefulness, and not faulty enough to permit it to be easily debugged.”*
- *“An interesting problem is posed by these two algorithms. The adequate scheme used two bits of control information (verify and alternation bits) per message while the inadequate scheme used only one bit (the acknowledge bit). In Section 3, three states were described for the received message, and the control bits of the next transmission encoded into the two control bits the total information concerning which of the three states held on reception. This leads to the conjecture that at least two control bits are required for any adequate scheme of this sort, and that only one control bit will never do. The reliable duplex transmission problem would, of course, have to be better formalized before it could be claimed that such a conjecture were ‘proven’.”*

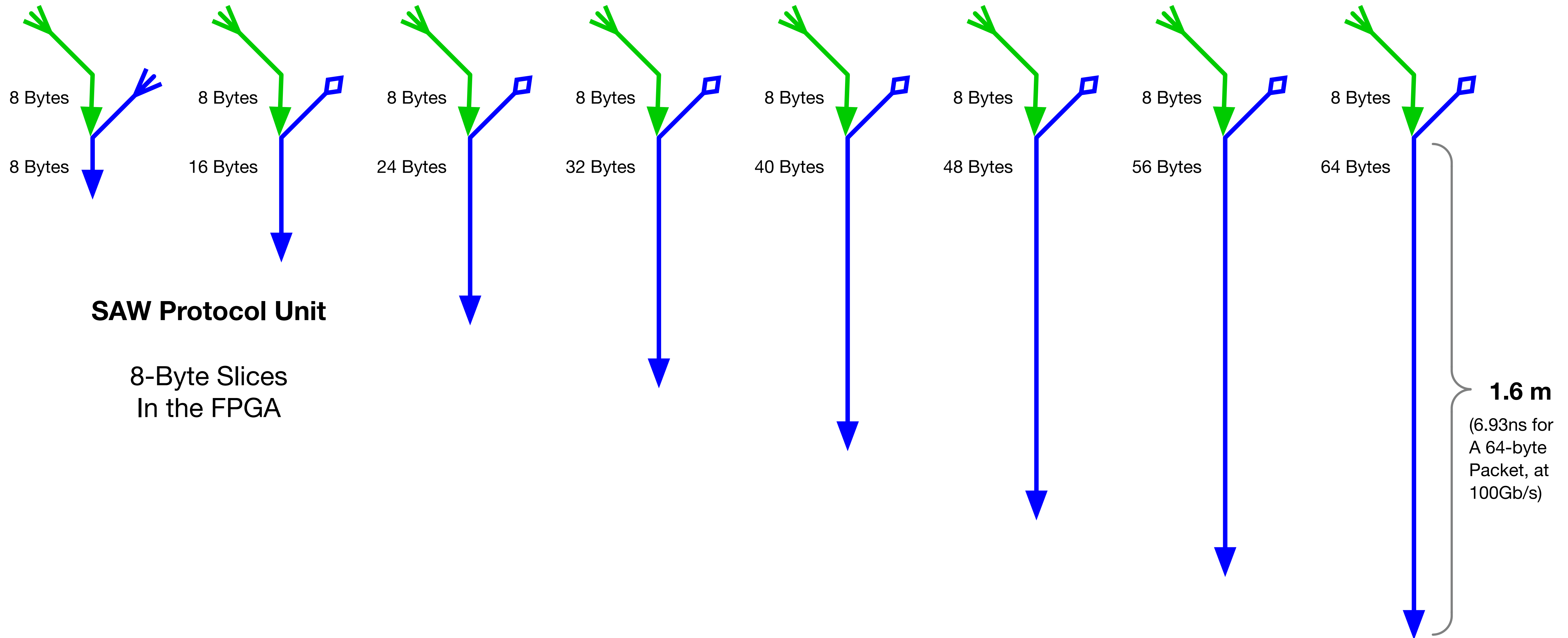
*ACM: [Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines](#).

Post Meeting Corrections

- [Robert Garner] My perspective (as was Metcalfe's) is that *it's not that Ethernet trades off higher performance for lower reliability, but instead pushes reliability up to the next higher level of protocols*. Metcalfe realized that different applications over Ethernet will need/want different degrees of reliability. For example, it may not be a disaster if an occasional network status inquiry or voice packet is lost (former retransmit, latter interpolate), *but would be a disaster if say a monetary or database transaction packet were lost*.
- Recall that Metcalfe's Ethernet founding memos/papers stated that the Ethernet provides "probabilistic delivery." *It's up to the next layer up to provide whatever degree/measure of reliability is required/desired*. Metcalfe brilliantly recognized that network implementations are not infallible, so it's counterproductive to try to make them perfect at the lowest transport level (with say an ACK mechanism). Thus, as you've seen, he (and John Shoch) began to formulate PUP at the same time Metcalfe and Boggs were implementing the first Ethernet hardware/firmware at Xerox PARC. (Then David Boggs and Ed Taft refined/implemented it more fully over several years.)
- Lynch's and Bartlett's 'verify and alternating-counter bits' protocol can be viewed as a predecessor of higher level protocols like TCP (with its sliding window version of flow control and its addition of non-reliable IP, to yield the merged TCP/IP.). UDP is frequently used when unreliable delivery is less of an issue.

*ACM: [Reliable Full-Duplex File Transmission over Half Duplex Telephone Lines](#).

Stop and wait (SAW) on the wire



8 Byte SerDes Slices “Time Intervals”

Slice Transmission Times From 1 Gigabit to 1 Terabit Ethernet (in ns)

Over-head	Slices	Slice Size (B)	Total (Bytes)	Total (Bits)	Bits after encoding					Total Bit Times (ns) (Assumes 1Gbit is 1ns per bit)					
						Ethernet Bit Transmission (from 1Gbit to 1 TBit)									
20		8			1.03125	1	10	25	40	50	100	200	400	800	1000
One Bit Time					1.00000	1.0000	0.1000	0.0400	0.0250	0.0200	0.0100	0.0050	0.0025	0.0013	0.0010
20	1	8	28	224	231.0	231.0	23.100	9.240	5.775	4.620	2.310	1.155	0.578	0.289	0.231
20	2	16	36	288	297.0	297.0	29.70	11.88	7.43	5.94	2.97	1.485	0.743	0.371	0.297
20	3	24	44	352	363.0	363.0	36.30	14.52	9.08	7.26	3.63	1.815	0.908	0.454	0.363
20	4	32	52	416	429.0	429.0	42.90	17.16	10.73	8.58	4.29	2.145	1.073	0.536	0.429
20	5	40	60	480	495.0	495.0	49.50	19.80	12.38	9.90	4.95	2.475	1.238	0.619	0.495
20	6	48	68	544	561.0	561.0	56.10	22.44	14.03	11.22	5.61	2.805	1.403	0.701	0.561
20	7	56	76	608	627.0	627.0	62.70	25.08	15.68	12.54	6.27	3.135	1.568	0.784	0.627
20	8	64	84	672	693.0	693.0	69.30	27.72	17.33	13.86	6.93	3.465	1.733	0.866	0.693
20	16	128	148	1184	1221.0	1,221.0	122.10	48.84	30.53	24.42	12.21	6.105	3.053	1.526	1.221

8 Byte SerDes Slices “Space Intervals”

Slice Transmission Distances (in m)

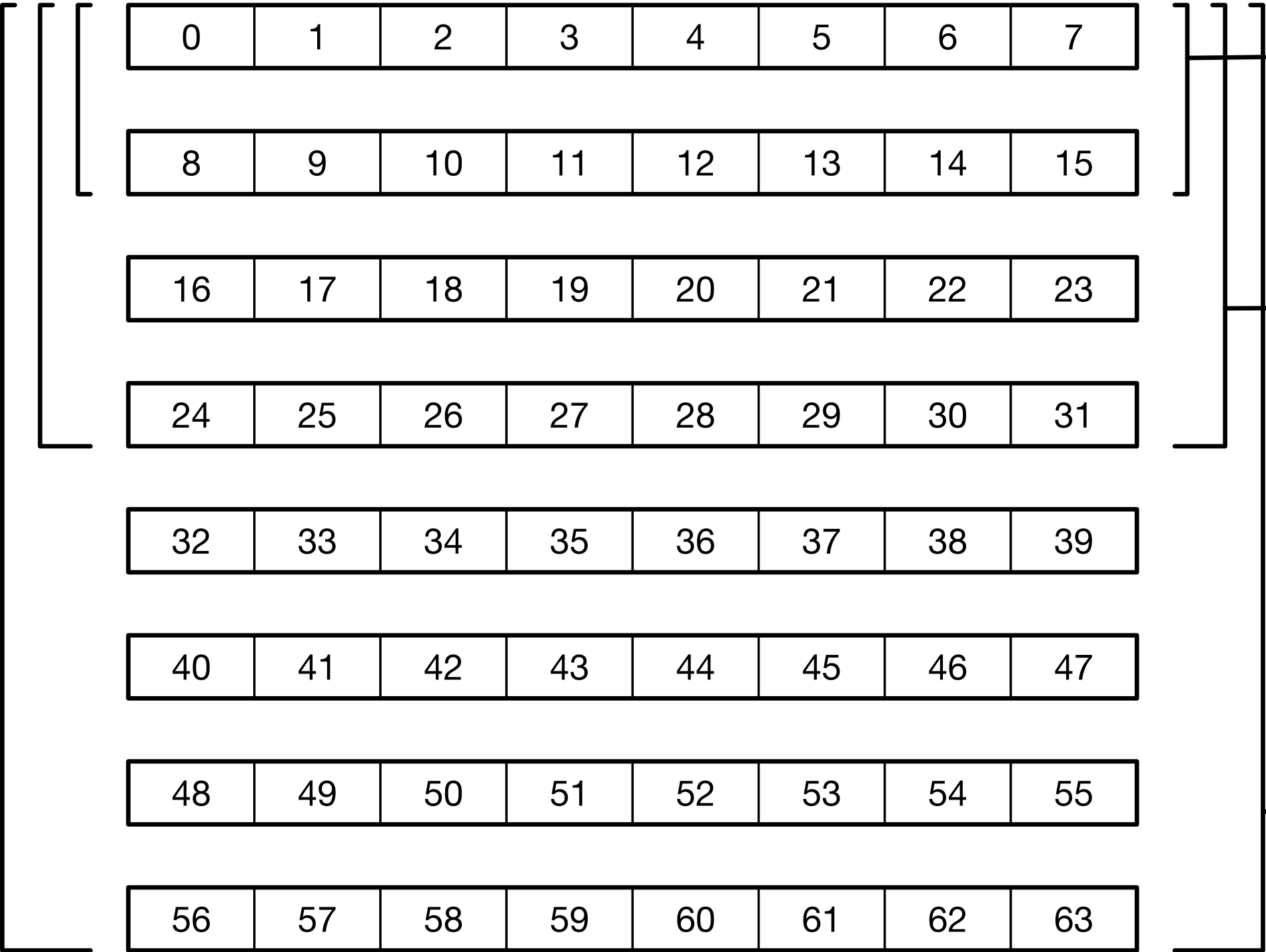
						SOL	ns Conversion	m conversion							
	Slices	Slice Size (B)	Total (Bytes)	Total (Bits)	Bits after encoding	2.3E+08	1E-09	1		Distance travelled for these bit times (in m)					
		8			1.03125	1	10	25	40	50	100	200	400	800	1000
One Bit distance					1.00000	0.23000	0.02300	0.00920	0.00575	0.00460	0.00230	0.00115	0.00058	0.00029	0.00023
	1	8	8	64	66	53.130	5.313	2.125	1.328	1.063	0.531	0.266	0.133	0.066	0.053
	2	16	16	128	132	68.310	6.831	2.732	1.708	1.366	0.683	0.342	0.171	0.085	0.068
	3	24	24	192	198	83.490	8.349	3.340	2.087	1.670	0.835	0.417	0.209	0.104	0.083
	4	32	32	256	264	98.670	9.867	3.947	2.467	1.973	0.987	0.493	0.247	0.123	0.099
	5	40	40	320	330	113.850	11.385	4.554	2.846	2.277	1.139	0.569	0.285	0.142	0.114
	6	48	48	384	396	129.030	12.903	5.161	3.226	2.581	1.290	0.645	0.323	0.161	0.129
	7	56	56	448	462	144.210	14.421	5.768	3.605	2.884	1.442	0.721	0.361	0.180	0.144
	8	64	64	512	528	159.390	15.939	6.376	3.985	3.188	1.594	0.797	0.398	0.199	0.159
	16	128	128	1024	1056	280.830	28.083	11.233	7.021	5.617	2.808	1.404	0.702	0.351	0.281

Register Transfer Model

Bits come into the SERDES and are available 8 bytes at a time to the FPGA Logic.
There is no need to wait for full size frames. The FPGA can pipeline 8 or 16 Bytes at a time

The FPGA processes 8 byte 'slices' per clock.
Use each slice as a fixed set of register pairs:
16 Bytes, 32 Bytes, and 64 Bytes. That's it.

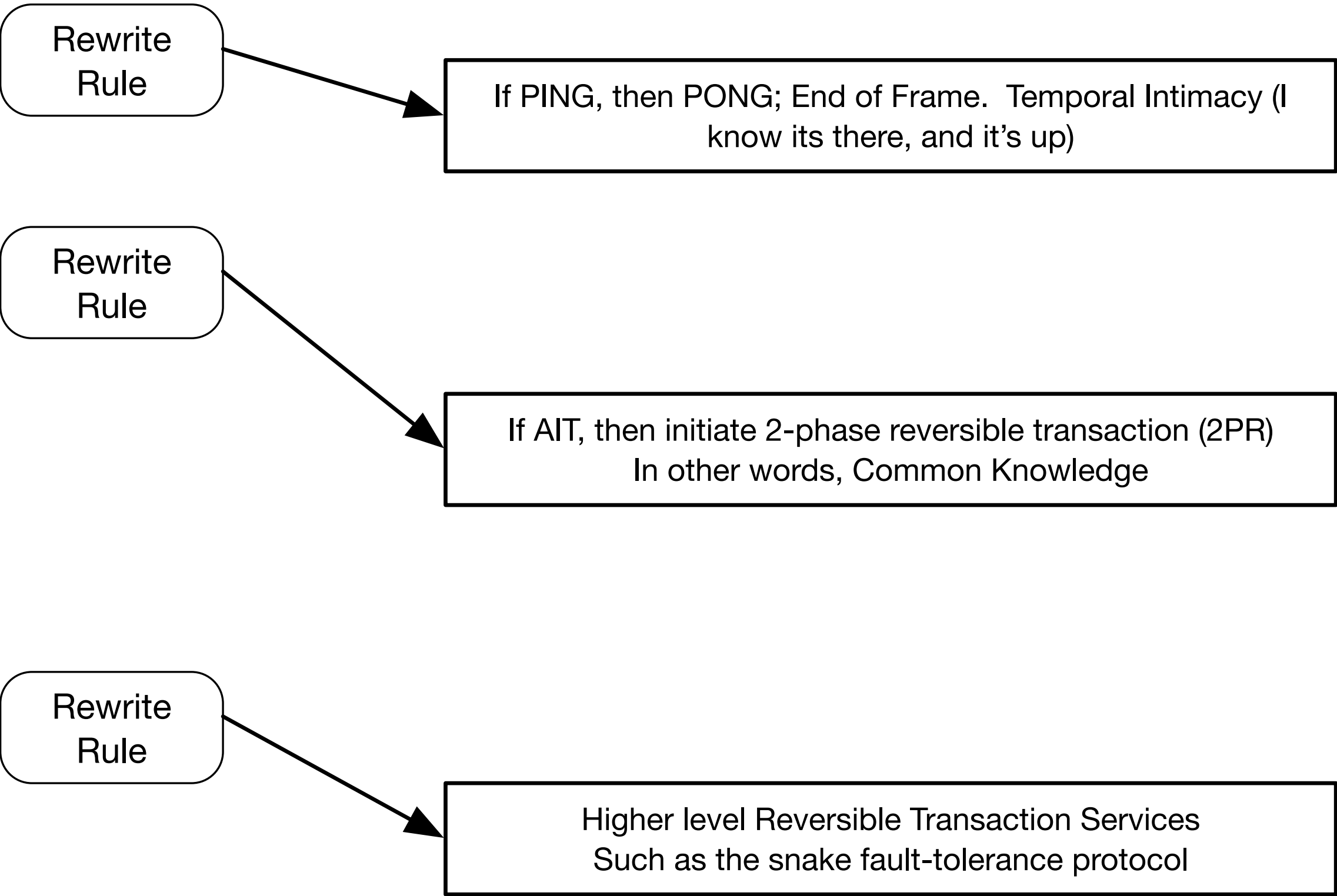
Register Interface (Bytes)



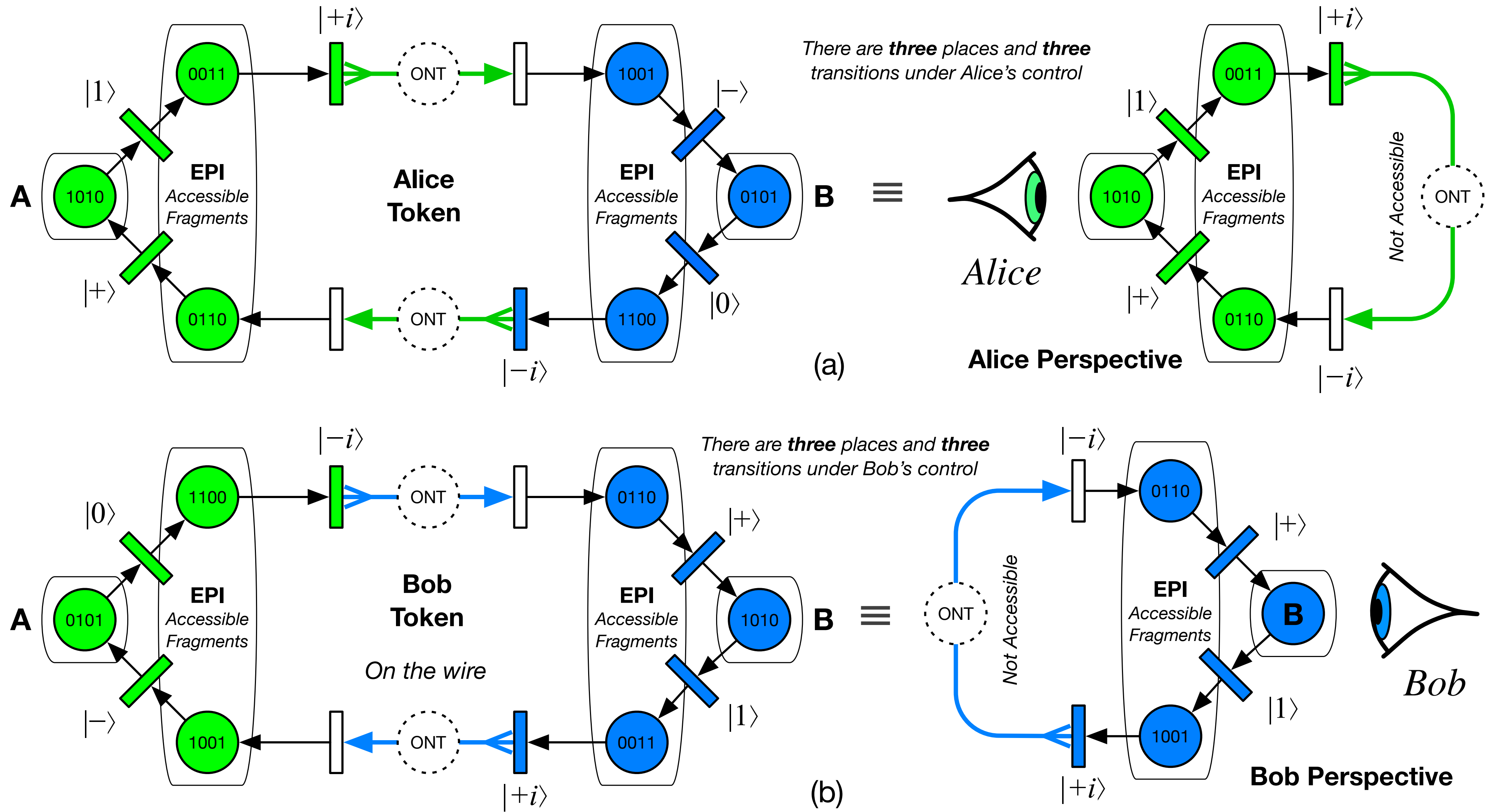
Software API: `ac_int`

https://github.com/hlslibs/ac_types/blob/master/include/ac_int.h

Immediate responses possible



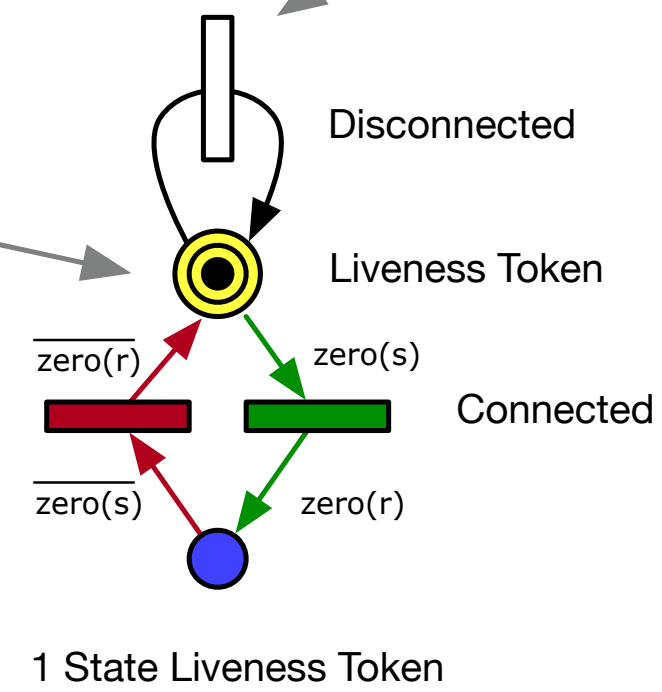
Petri net Specification



State Machines

FPGA Sice 1:

Black Token
Carries no
information.
(Liveness only)



There can be more than one token
carrying “concurrent” information
in the same clock cycle.

Petri-Nets are not “state machines”, they
can have “concurrently active” tokens
driving the execution

Alice Token

Bob Token

PROTOCOL	LIVENESS	MACHINE	TRANSITION	PROTOCOL	LIVENESS	MACHINE	TRANSITION
A1	A2	A3	A4	B1	B2	B3	B4

BLACK

BLUE

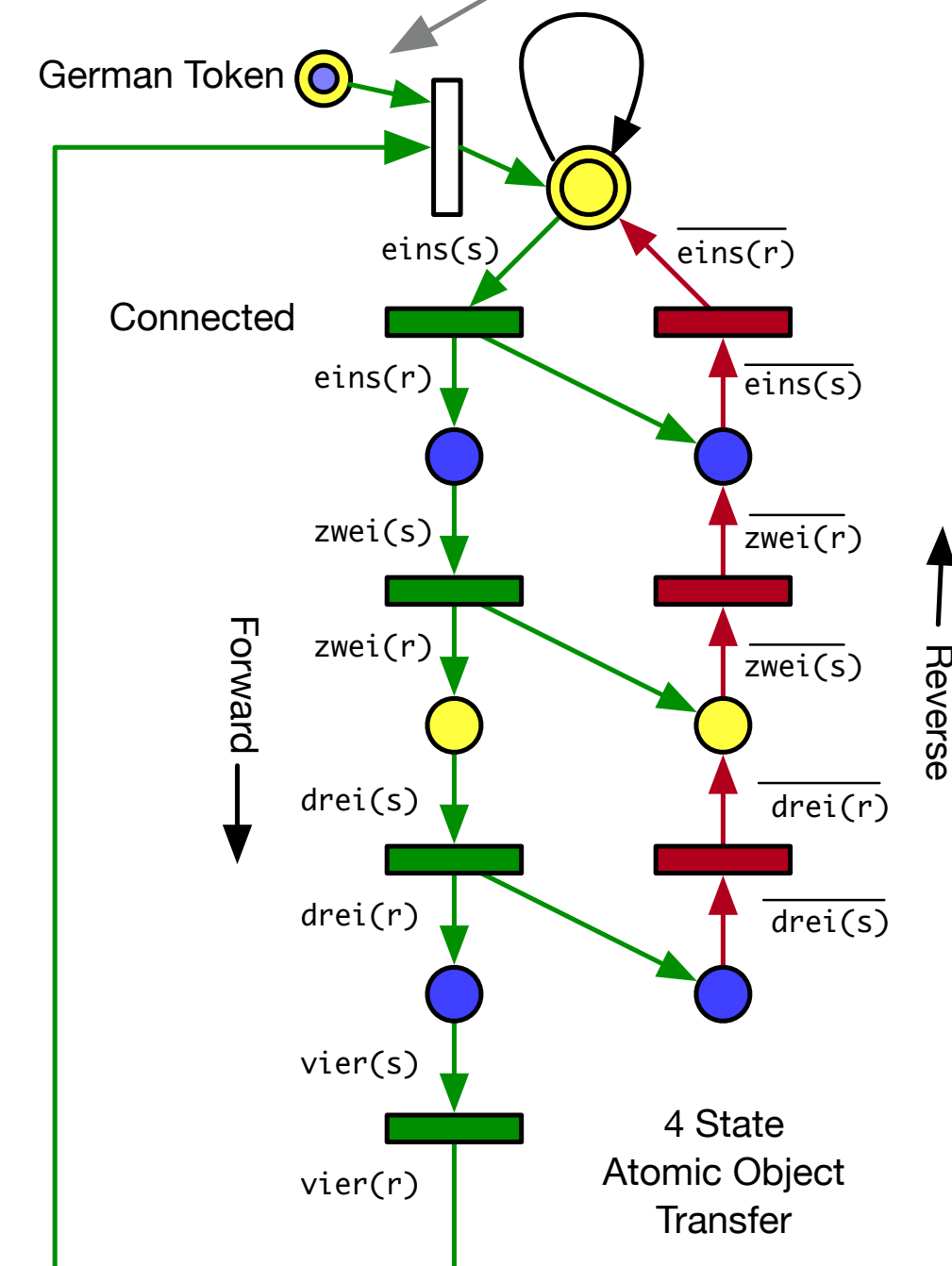
One Thread?

End of Clock
(6.25ns)

Next Clock
(6.25ns)

Next Clock
(6.25ns)

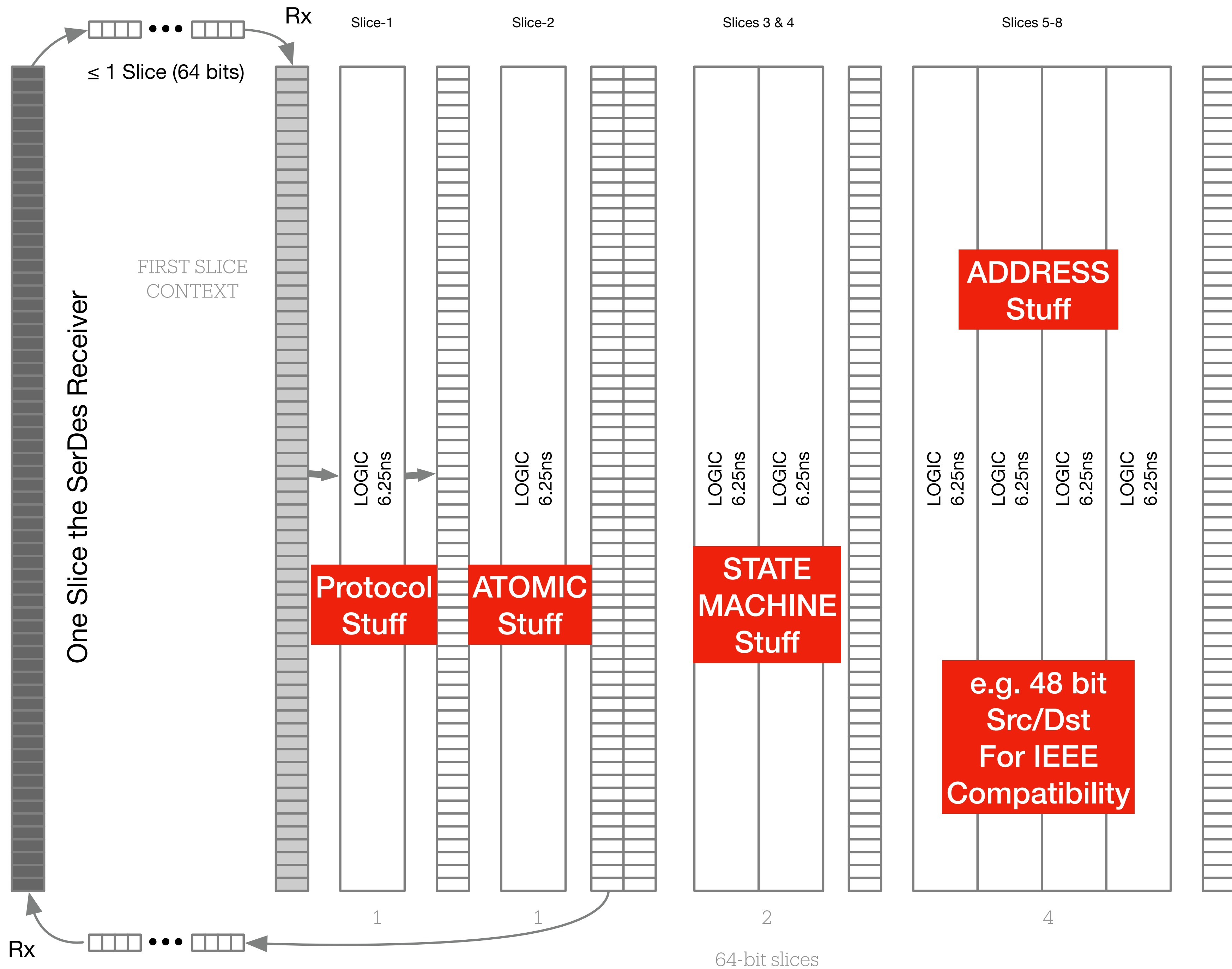
Blue Token
Carries information
with 4-state protocol



Reverse

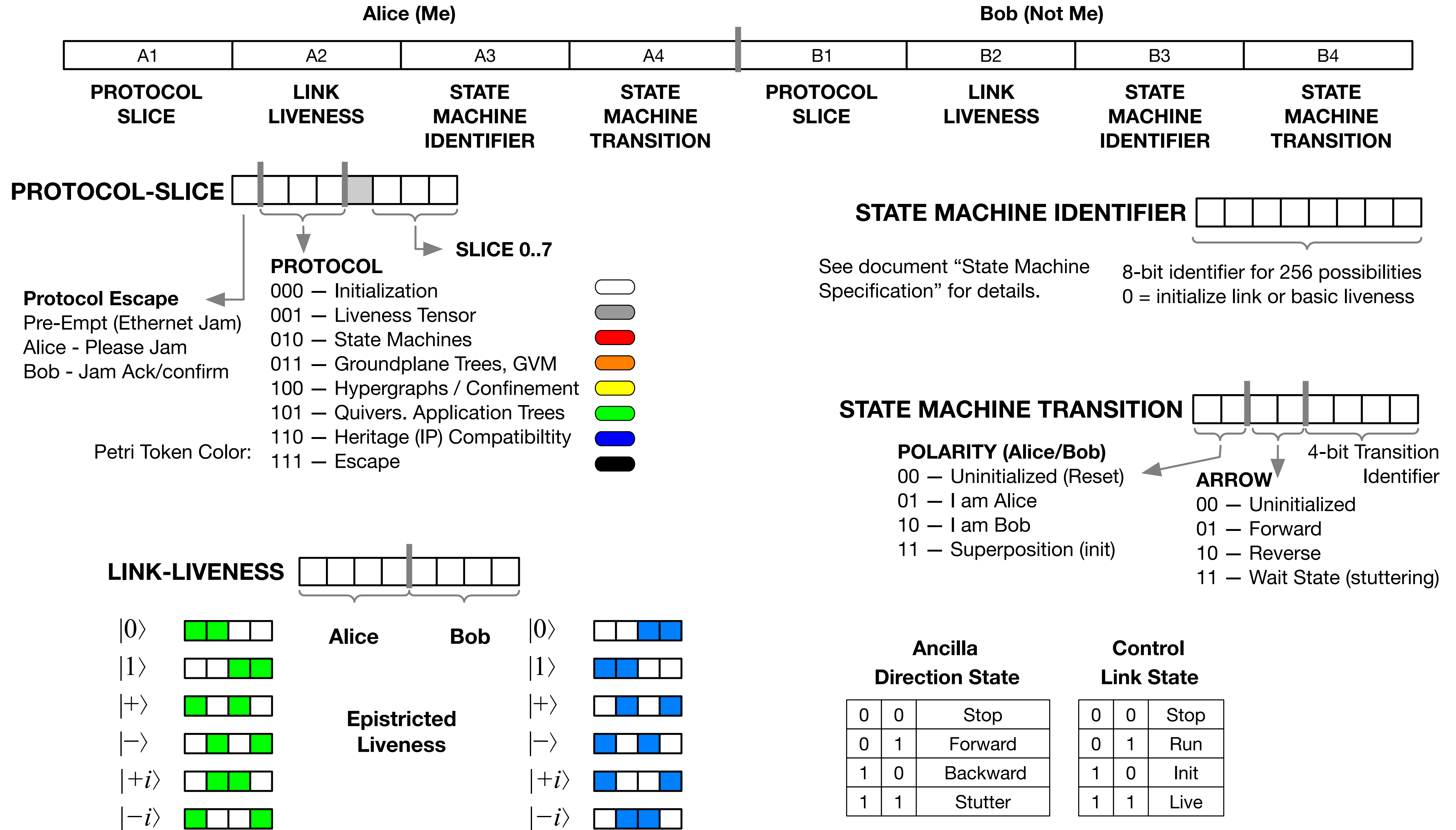
Forward

Bits On the Wire



Bits On the Wire

Slice Engine



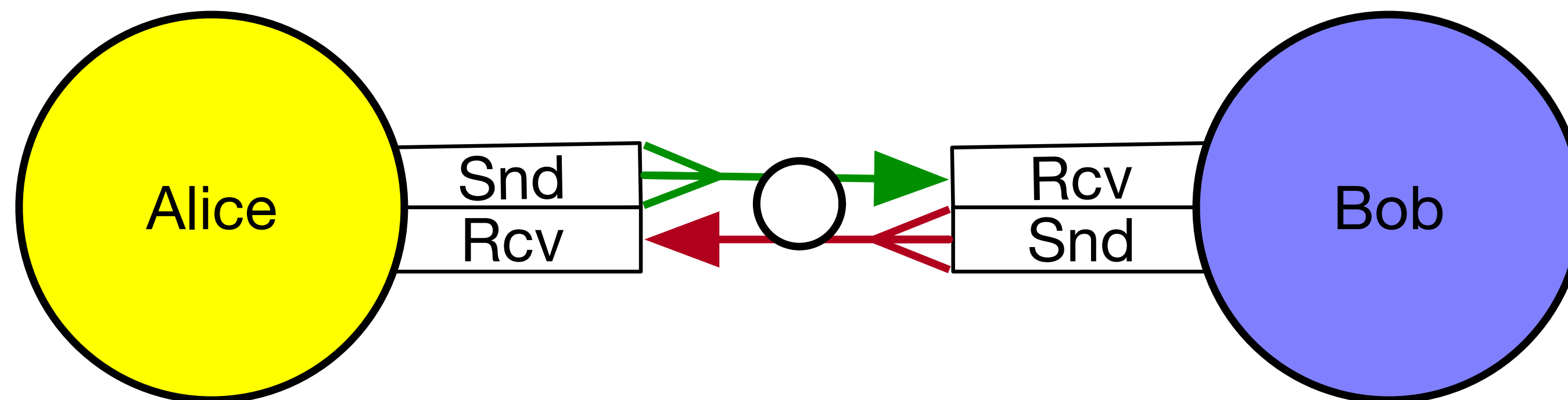
Discussion

Please Raise your Hand,
Turn on your Microphone
(and Video if Possible)

Spare Slides

Not shown at this meeting

Bits On the Wire



Green: Forward Evolution



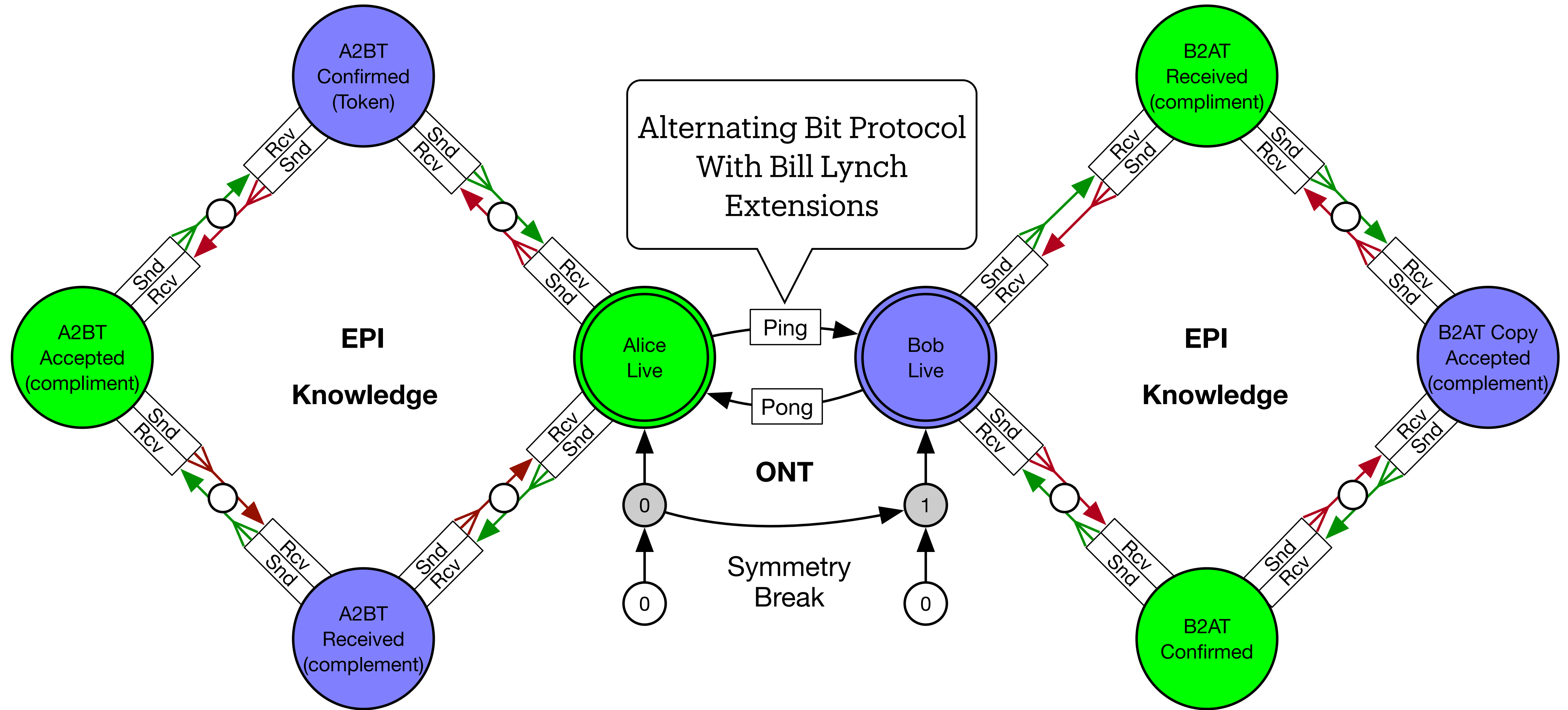
Forward Progress (+1)

Red: Reverse Evolution



Reverse Progress (-1)

State Machine



Problem: Network Partitions:

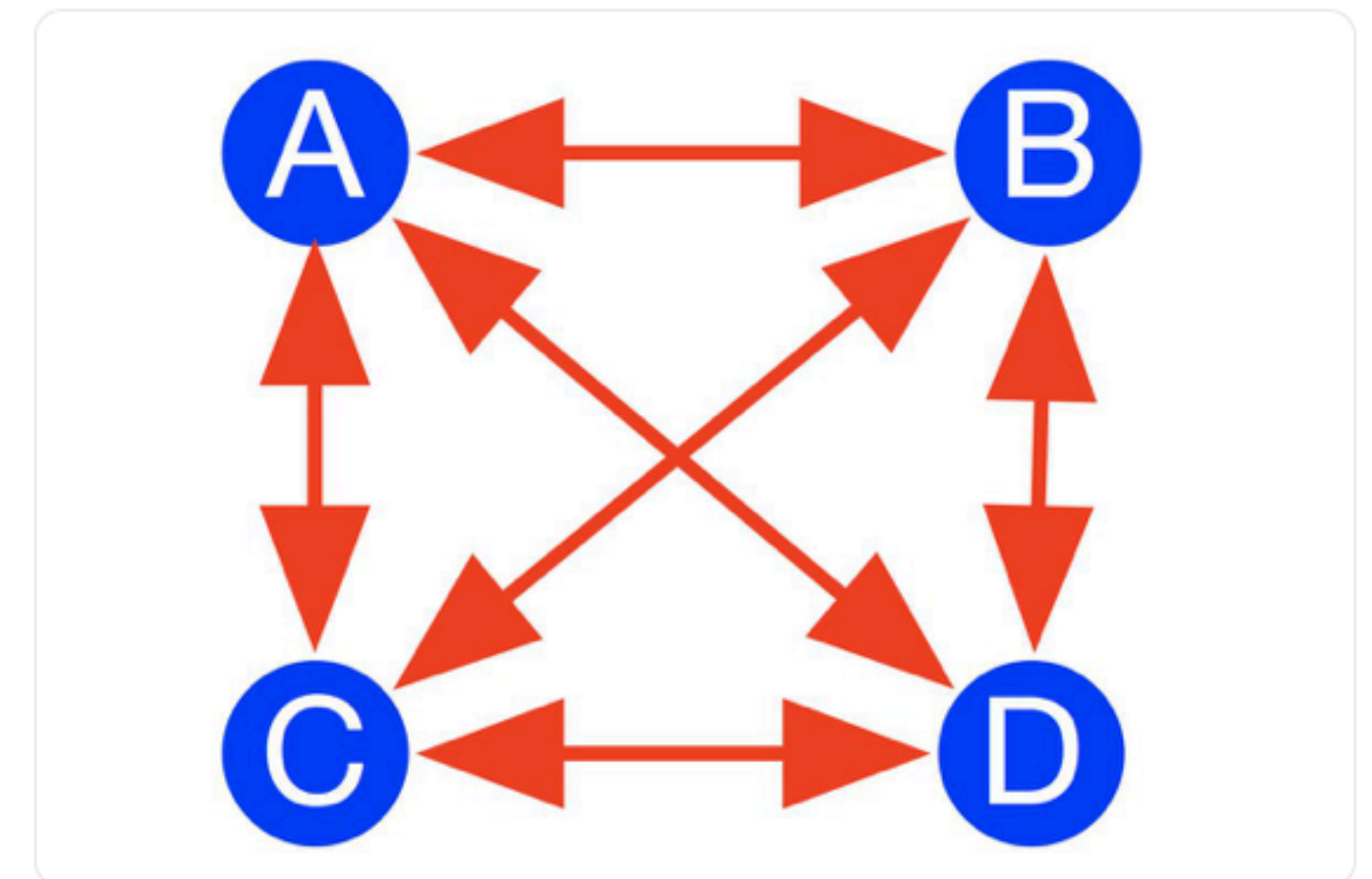
- Link failures are invisible (hidden) in a Clos
- They are 100% visible to us in a mesh
- For 4 nodes, there are $(n(n - 1)/2) = 6$ links, 4 configurations on each link (00,01,10,11); gives $6^4 - 1 = 1295$ possible failure modes
- And that's only the *clean* (binary) failures
- *Flakey connections are much worse*
- DÆ protocol is designed to be exquisitely sensitive to packet loss and corruption
- We monitor, detect, diagnose link failures, and recover *reversibly* and *automatically*



Colm MacCárthaigh
@colmmacc

Follow

In distributed systems, consensus protocols etc, it's easy to miss how varied network partitions can be. They don't have to be symmetrical, so A can see B, but B can't see A. In a 4 node setup, there are over 1200 potential partitions, and a flaky network can reproduce them all.



7:44 AM - 3 Jul 2019

70 Retweets 203 Likes



What Problem are we solving?

Link Failures are the #1 Cause of failures in large LLM Installations

- SSD's are #2
- GPUs fail quite spectacularly
- CPUs also fail
- Memory Errors

See: [Fire-Flyer HI-HPC: A Cost-Effective Software-Hardware Co-Design for Deep Learning*](#)

Standard Solution is Take snapshots so we can recover from the snapshot instead of going back days to the beginning of the run

This is an old problem - Snapshots can never be atomic cross a distributed system — Without Atomicity and in-order delivery

* <https://arxiv.org/abs/2408.14158>

Network Partitions

- Link failures are invisible (hidden) in a Clos
- They are 100% visible to us in a mesh
- For 4 nodes, there are $(n(n-1)/2) = 6$ links, 4 configurations on each link (00,01,10,11); gives $6^4 - 1 = 1295$ possible failure modes
 - And that's only the *clean* (binary) failures
 - *Flakey connections are much worse*
- DÆ protocol is designed to be exquisitely sensitive to packet loss and corruption
- We monitor, detect, diagnose link failures, and recover *reversibly* and *automatically*

DÆ

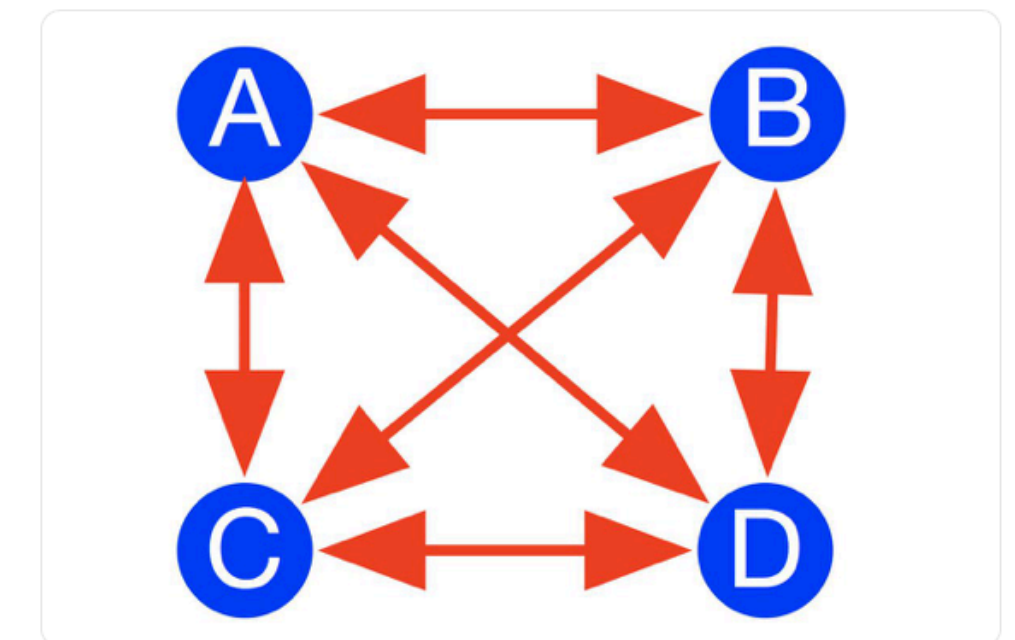
DÆDÆLUS



Colm MacCárthaigh
@colmmacc

Follow

In distributed systems, consensus protocols etc, it's easy to miss how varied network partitions can be. They don't have to be symmetrical, so A can see B, but B can't see A. In a 4 node setup, there are over 1200 potential partitions, and a flaky network can reproduce them all.



7:44 AM - 3 Jul 2019

70 Retweets 203 Likes



Agenda (Mon 8AM)

Resilience and Security

Alan Karp

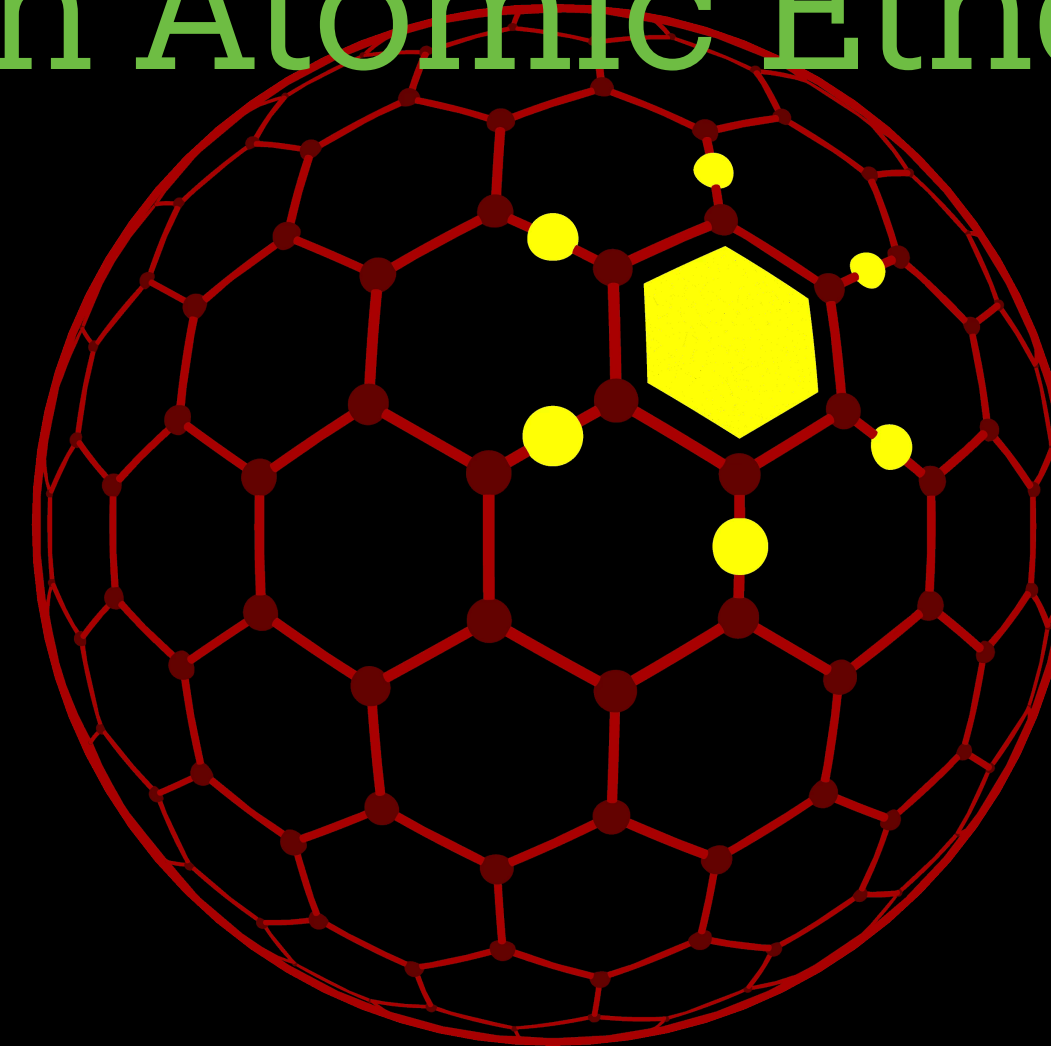
Threat Model

The first thing you have to do for security is create a threat model. That involves deciding what resources you need to protect, the value of those resources, and the ways they may be compromised. Typically, overhead increases with the capabilities of the assumed adversary.

The first thing you have to do for resilience is decide what are "tolerated" failures, ones that you guarantee to recover from, and "untolerated" failures that you don't. An example of the former is a broken link; an example of the latter is a comet destroying all life on Earth. Typically, overhead increases with the number of components involved in a tolerated failure.

Let's try to get a handle on the threat model in the first meeting and discuss tolerated failures as time allows.

Open Atomic Ethernet



Begin	End	Time	Workgroup Topic (Monday 02025-MAR-10 @ 8am)
8:00 AM	8:05 AM	5m	Group Discussion Led by Alan Karp
8:05 AM	8:10 AM	5m	Participant 1:
8:10 AM	8:15 AM	5m	Participant 2:
8:15 AM	8:20 AM	5m	Participant 3:
8:20 AM	8:25 AM	5m	Participant 4:
8:25 AM	8:30 AM	5m	Participant 5:

Agenda (Thurs 8AM)

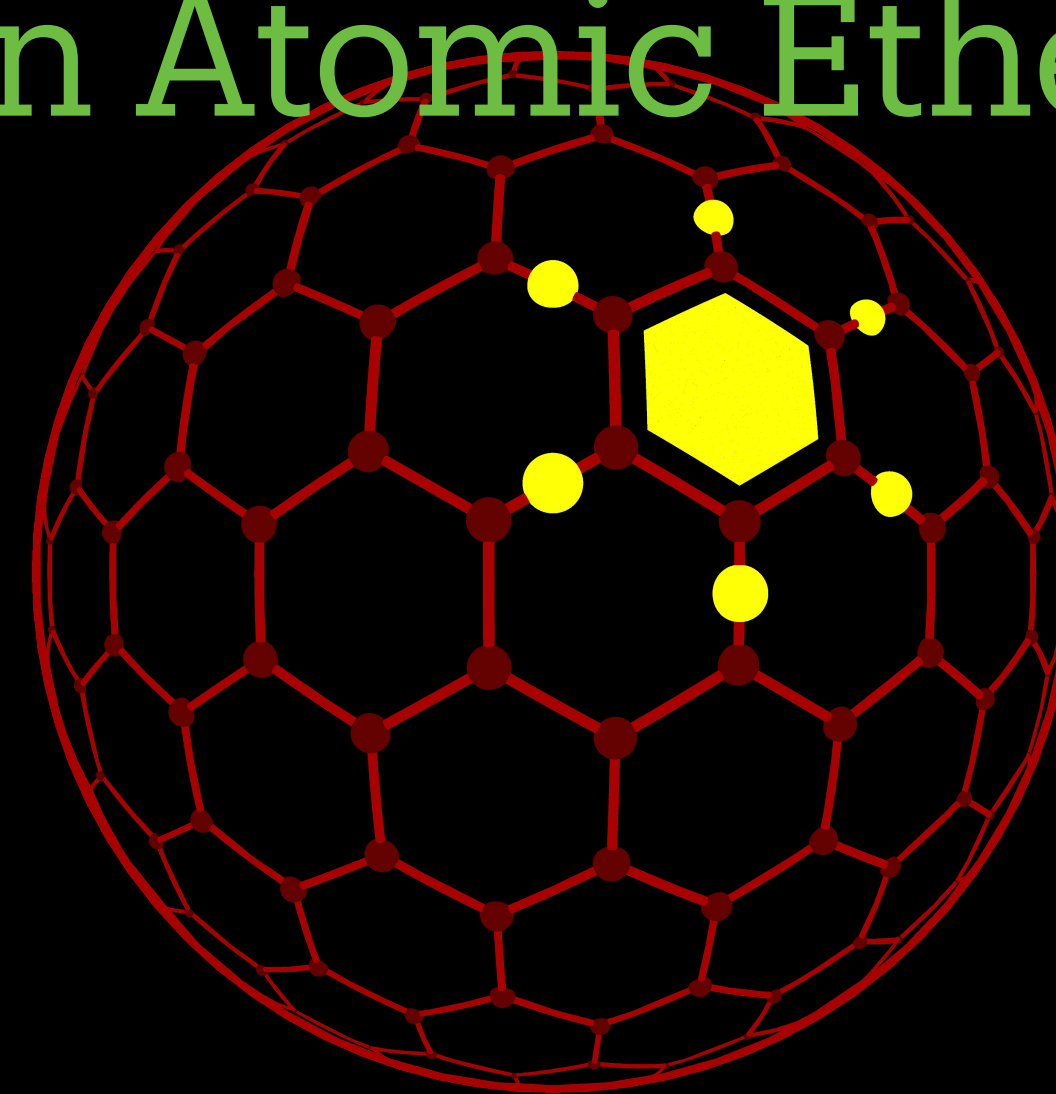
FPGA Implementation
Edmund Humenburger

Ongoing discussions of
requirements

1. Group discussion led by Edmund

If more than 5 people want to
present this week, then the next
week will be used too to present.

Open Atomic Ethernet



Begin	End	Time	Workgrouup Topic (Tomorrow, Thursday 02025-MAR-06 @ 8am)
8:00 AM	8:05 AM	5m	Introduction — 5 participants can present:
8:05 AM	8:10 AM	5m	Participant 1:
8:10 AM	8:15 AM	5m	Participant 2:
8:15 AM	8:20 AM	5m	Participant 3:
8:20 AM	8:25 AM	5m	Participant 4:
8:25 AM	8:30 AM	5m	Participant 5: